

09a — ORMs (Object-Relational Mappers)

Key Concepts

Term	Definition
ORM	Library that maps DB tables to code objects — no raw SQL needed for basic operations
Full ORM	Generates SQL automatically (e.g., Entity Framework)
Micro-ORM	You write SQL, it maps results to objects (e.g., Dapper)
Lazy Loading	Related data fetched on demand — triggers extra queries
Eager Loading	Related data fetched upfront in the same query
Migration	ORM-managed versioned schema changes

How It Works

1. Define a model/entity class → maps to a DB table
2. Properties → map to columns
3. Call ORM methods (`.Find()`, `.Where()`, `.Include()`)
4. ORM generates and executes SQL
5. Results returned as objects

Full ORM vs Micro-ORM

	Entity Framework (Full ORM)	Dapper (Micro-ORM)
SQL	Auto-generated	You write it
Results	Mapped to objects	Mapped to objects
Best for	CRUD, fast development	Complex queries, performance-critical paths
Risk	Inefficient SQL, N+1	None — you control the SQL

Popular ORMs

Language	ORM
.NET / C#	Entity Framework Core, Dapper
Python	SQLAlchemy, Django ORM
Java	Hibernate
Node.js	Prisma, Sequelize, TypeORM

Language	ORM
Ruby	ActiveRecord

Industry Examples

- **GitHub** — ActiveRecord (Rails ORM) for repos, issues, PRs
- **Instagram** — Django ORM early on; dropped to raw SQL for performance at scale
- **Shopify** — ActiveRecord for standard ops; raw SQL for reporting

Interview Questions

Question	One-Line Answer
What is an ORM?	Library that maps DB tables to code objects — query without raw SQL
What are ORM downsides?	Can generate inefficient SQL, hides query logic, causes N+1 if misused
EF vs Dapper?	EF generates SQL automatically; Dapper takes raw SQL but maps results to objects
When NOT to use ORM?	Complex joins, bulk operations, performance-critical paths — use raw SQL instead
Have you used ORMs?	EF Core and Dapper in .NET; Dapper for performance-sensitive queries